31.1 配置加载与解析

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~160k tokens，输出 ~8k tokens（本节）

配置系统是 OpenClaw 的"神经中枢"——从 Agent 行为到通道连接，从模型选择到安全策略，几乎所有运行时行为都由一个 JSON5 配置文件控制。本节深入分析配置的加载流水线：从磁盘读取原始文件到最终产出经过验证、替换、默认值填充的 `OpenClawConfig` 对象，这中间经历了哪些步骤。

31.1.1 配置文件位置： `~/.openclaw/openclaw.json` （JSON5 格式）

默认路径

OpenClaw 的配置文件默认存储在用户主目录下的 `.openclaw` 目录：

```
~/.openclaw/openclaw.json
```

> **衍生解释**：JSON5 是 JSON 的超集，允许注释（ `//` 和 `/* */` ）、尾随逗号、单引号字符串、十六进制数字等。对于需要人工编辑的配置文件，JSON5 比严格的 JSON 友好得多——你可以写注释来解释配置项的含义。OpenClaw 使用 `json5` npm 包来解析配置文件。

路径解析策略

配置路径的解析并不简单——OpenClaw 需要兼容多代产品名称的历史遗留，支持环境变量覆盖，并处理多个候选位置：

```
// src/config/paths.ts（简化）

const LEGACY_STATE_DIRNAMES = [".clawdbot", ".moltbot", ".moldbot"];
const NEW_STATE_DIRNAME = ".openclaw";
const CONFIG_FILENAME = "openclaw.json";
const LEGACY_CONFIG_FILENAMES = ["clawdbot.json", "moltbot.json", "moldbot.json"];

// 状态目录解析（优先级从高到低）
export function resolveStateDir(env, homedir): string {
  // 1. 环境变量显式指定
  if (env.OPENCLAW_STATE_DIR) return resolveUserPath(env.OPENCLAW_STATE_DIR);

  // 2. 新目录存在 → 使用新目录
  if (fs.existsSync(newStateDir(homedir))) return newStateDir(homedir);

  // 3. 旧目录存在 → 使用旧目录（向后兼容）
  const legacy = legacyDirs.find(dir => fs.existsSync(dir));
  if (legacy) return legacy;

  // 4. 都不存在 → 使用新目录
  return newStateDir(homedir);
}
```

配置文件的搜索顺序更加复杂——`resolveDefaultConfigCandidates` 会生成一个候选列表：

优先级	候选路径	来源
1	`$OPENCLAW_CONFIG_PATH`	环境变量显式指定
2	`$OPENCLAW_STATE_DIR/openclaw.json`	自定义状态目录
3	`$OPENCLAW_STATE_DIR/clawdbot.json` 等	自定义目录下的旧文件名
4	`~/.openclaw/openclaw.json`	新默认位置
5	`~/.openclaw/clawdbot.json` 等	新目录下的旧文件名
6	`~/.clawdbot/clawdbot.json` 等	旧默认位置

系统会选择第一个实际存在的候选文件。这种设计确保了无论用户从哪个版本升级，都不需要手动迁移配置文件路径。

关键环境变量

环境变量	作用	默认值
`OPENCLAW_CONFIG_PATH`	指定配置文件完整路径	无
`OPENCLAW_STATE_DIR`	指定状态目录（配置文件在其下）	`~/.openclaw`
`OPENCLAW_NIX_MODE`	Nix 模式（只读配置，禁用自动安装）	无
`OPENCLAW_GATEWAY_PORT`	覆盖 Gateway 监听端口	18789

31.1.2 配置 Schema（`src/config/schema.ts`）

JSON Schema 生成

OpenClaw 的配置 Schema 由 Zod 模式自动生成 JSON Schema，供 Web UI 的配置编辑器和外部工具使用：

```
// src/config/schema.ts（简化）

export type ConfigSchema = ReturnType<typeof OpenClawSchema.toJSONSchema>;

export type ConfigSchemaResponse = {
  schema: ConfigSchema;         // 完整 JSON Schema
  uiHints: ConfigUiHints;       // UI 提示（分组、排序、敏感标记等）
  version: string;              // OpenClaw 版本
  generatedAt: string;          // 生成时间
};
```

UI 分组与排序

Schema 不仅定义了配置的结构，还为 Web UI 提供了丰富的展示提示：

```
const GROUP_LABELS: Record<string, string> = {
  gateway: "Gateway",
  agents: "Agents",
  tools: "Tools",
  models: "Models",
  channels: "Messaging Channels",
  hooks: "Hooks",
  skills: "Skills",
  plugins: "Plugins",
  // ...
};

export type ConfigUiHint = {
  label?: string;          // 显示标签
  help?: string;           // 帮助文本
  group?: string;          // 所属分组
  order?: number;          // 排序权重
  advanced?: boolean;      // 是否为高级选项
  sensitive?: boolean;     // 是否为敏感字段（如 API Key）
  placeholder?: string;    // 占位符文本
};
```

`sensitive: true` 的字段（如 Token、API Key）在 UI 中会被掩码显示，且在日志中会被自动脱敏。

31.1.3 Zod Schema 校验（`src/config/zod-schema.ts`）

Zod 运行时类型校验

OpenClaw 使用 Zod 库来定义和校验配置的运行时类型。这不是简单的 JSON Schema——Zod 可以在 TypeScript 编译时提供类型推断，同时在运行时执行精确的数据验证。

> **衍生解释**：Zod 是一个 TypeScript-first 的数据校验库。传统做法是先写 TypeScript 类型，再写独立的校验逻辑，两者容易不同步。Zod 让你只写一次 Schema，同时得到编译时类型和运行时校验。`z.object({ name: z.string(), port: z.number().int().positive() })` 这个 Schema 既是 TypeScript 类型定义，又是运行时校验器。

Zod Schema 的定义按功能模块分散在多个文件中：

Schema 文件	覆盖范围
`zod-schema.core.ts`	颜色、模型配置
`zod-schema.agents.ts`	Agent 列表、音频、绑定
`zod-schema.agent-defaults.ts`	Agent 默认配置
`zod-schema.agent-runtime.ts`	工具配置
`zod-schema.channels.ts`	通道配置
`zod-schema.session.ts`	会话、消息、命令
`zod-schema.providers.ts`	通道提供者（WhatsApp 等）
`zod-schema.approvals.ts`	审批配置

最终在 `zod-schema.ts` 中组合为完整的 `OpenClawSchema`：

```
// src/config/zod-schema.ts（简化）

import { z } from "zod";

export const OpenClawSchema = z.object({
  gateway: GatewaySchema.optional(),
  agents: AgentsSchema.optional(),
  tools: ToolsSchema.optional(),
  models: ModelsConfigSchema.optional(),
  channels: ChannelsSchema.optional(),
  hooks: z.object({
    enabled: z.boolean().optional(),
    token: z.string().optional(),
    internal: InternalHooksSchema.optional(),
    mappings: z.array(HookMappingSchema).optional(),
    // ...
  }).optional(),
  session: SessionSchema.optional(),
  messages: MessagesSchema.optional(),
  // ... 数十个顶级键
}).strict();  // strict() 拒绝未知键
```

`.strict()` 的使用很重要——它确保配置文件中不存在拼写错误或过时的键。如果用户写了 `gateway.tokne`（拼错了 `token`），Zod 会立即报错而不是静默忽略。

校验流程

```
// src/config/validation.ts（简化）

export function validateConfigObject(raw: unknown) {
  // 1. 先检查是否存在遗留配置（如 `whatsapp` → `channels.whatsapp`）
  const legacyIssues = findLegacyConfigIssues(raw);
  if (legacyIssues.length > 0) return { ok: false, issues: legacyIssues };

  // 2. Zod Schema 校验
  const validated = OpenClawSchema.safeParse(raw);
  if (!validated.success) return { ok: false, issues: validated.error.issues };

  // 3. 语义校验：Agent 目录不能重复
  const duplicates = findDuplicateAgentDirs(validated.data);
  if (duplicates.length > 0) return { ok: false, issues: [...] };

  // 4. 语义校验：头像路径必须在工作区内
  const avatarIssues = validateIdentityAvatar(validated.data);
  if (avatarIssues.length > 0) return { ok: false, issues: avatarIssues };

  // 5. 通过后应用默认值
  return { ok: true, config: applyDefaults(validated.data) };
}
```

校验是多层的：Zod 负责结构校验（类型、范围、必填），自定义逻辑负责语义校验（业务规则、跨字段一致性）。

31.1.4 配置 IO（`src/config/io.ts`）

加载流水线

`loadConfig` 是整个配置系统的入口——它编排了从磁盘读取到最终产出的完整流水线：

```typescript
// src/config/io.ts（简化）

function loadConfig(): OpenClawConfig {
  // ① 读取原始 JSON5 文件
  const raw = fs.readFileSync(configPath, "utf-8");
  const parsed = JSON5.parse(raw);

  // ② 解析 $include 指令（配置文件拆分与组合）
  const resolved = resolveConfigIncludes(parsed, configPath);

  // ③ 应用 config.env 到 process.env
  applyConfigEnv(resolved, process.env);

  // ④ 替换 ${VAR} 环境变量引用
  const substituted = resolveConfigEnvVars(resolved, process.env);

  // ⑤ 检查常见拼写错误
  warnOnConfigMiskeys(substituted);

  // ⑥ 检查 Agent 目录重复
  findDuplicateAgentDirs(substituted);

  // ⑦ Zod Schema 校验 + 插件校验
  const validated = validateConfigObjectWithPlugins(substituted);

  // ⑧ 版本警告（配置来自更新版本）
  warnIfConfigFromFuture(validated.config);

  // ⑨ 应用多层默认值
  const cfg = applyModelDefaults(
    applyCompactionDefaults(
      applyContextPruningDefaults(
        applyAgentDefaults(
          applySessionDefaults(
            applyLoggingDefaults(
              applyMessageDefaults(validated.config)))))));

  // ⑩ 路径规范化
  normalizeConfigPaths(cfg);

  // ⑪ Shell 环境变量回退
  loadShellEnvFallback({ enabled, env, expectedKeys });
```

```
  // ⑫ 应用运行时覆盖
  return applyConfigOverrides(cfg);
}
```

这个 12 步流水线反映了配置加载的复杂性——每一步都解决一个具体问题。

$include 指令

OpenClaw 支持配置文件拆分。一个大型配置可以被拆成多个文件：

```
// ~/.openclaw/openclaw.json
{
  "$include": ["./channels.json", "./agents.json"],
  "gateway": { "port": 18789 }
}
```

`resolveConfigIncludes` 在 JSON5 解析后、环境变量替换前执行，递归合并所有 `$include` 引用的文件。

配置缓存

频繁读取配置文件（尤其在高并发场景）会成为性能瓶颈。OpenClaw 内置了配置缓存机制：

```
// src/config/io.ts

const DEFAULT_CONFIG_CACHE_MS = 200;  // 200ms 缓存窗口

export function loadConfig(): OpenClawConfig {
  const now = Date.now();
  if (shouldUseConfigCache(process.env)) {
    if (cached && cached.configPath === configPath && cached.expiresAt > now) {
      return cached.config;  // 缓存命中
    }
  }
  const config = io.loadConfig();
  configCache = { configPath, expiresAt: now + cacheMs, config };
  return config;
}
```

默认缓存 200ms——在这个时间窗口内，多次 `loadConfig()` 调用只会读取一次磁盘。通过 `OPENCLAW_CONFIG_CACHE_MS` 环境变量可以调整，设为 `0` 则禁用缓存。

原子写入

配置文件的写入采用了经典的"写临时文件 → 原子重命名"策略：

```
// src/config/io.ts（简化）

async function writeConfigFile(cfg: OpenClawConfig) {
  // 1. 校验
  validateConfigObjectWithPlugins(cfg);

  // 2. 创建目录（权限 700：仅所有者可读写执行）
  await fs.mkdir(dir, { recursive: true, mode: 0o700 });

  // 3. 写入临时文件（权限 600：仅所有者可读写）
  const tmp = path.join(dir, `${basename}.${process.pid}.${randomUUID()}.tmp`);
  await fs.writeFile(tmp, json, { encoding: "utf-8", mode: 0o600 });

  // 4. 轮转备份（最多保留 5 份）
  await rotateConfigBackups(configPath);

  // 5. 原子重命名（POSIX）或 copy+unlink（Windows）
  await fs.rename(tmp, configPath);
}
```

写入流程中的三个安全措施：

1. **文件权限**：目录 `0o700` ，文件 `0o600` ——只有所有者能访问，防止其他用户读取包含 API Key 的配置。
2. **备份轮转**：每次写入前，保存最多 5 份历史备份（ `openclaw.json.bak` 、 `openclaw.json.bak.1` … `openclaw.json.bak.4` ），方便意外覆盖时恢复。
3. **原子重命名**：先写临时文件再重命名，避免写入过程中崩溃导致配置文件损坏。Windows 上如果 `rename` 失败（权限问题），降级为 `copyFile` + `chmod` 。

31.1.5 配置路径解析（ `src/config/config-paths.ts` ）

点号路径系统

OpenClaw 的配置路径使用点号分隔的路径表达式来定位嵌套值——类似 JavaScript 的属性访问链：

```
// src/config/config-paths.ts

export function parseConfigPath(raw: string): { ok: boolean; path?: string[] } {
  const parts = raw.trim().split(".").map(p => p.trim());
  // 安全检查：阻止 __proto__、prototype、constructor
  if (parts.some(part => BLOCKED_KEYS.has(part))) {
    return { ok: false, error: "Invalid path segment." };
  }
  return { ok: true, path: parts };
}
```

路径解析支持三种操作：

```
// 读取
getConfigValueAtPath(config, ["agents", "defaults", "model", "primary"]);
// → config.agents.defaults.model.primary

// 设置（自动创建中间节点）
setConfigValueAtPath(config, ["hooks", "internal", "enabled"], true);
// → config.hooks.internal.enabled = true

// 删除（自动清理空的父节点）
unsetConfigValueAtPath(config, ["hooks", "internal", "entries", "soul-evil"]);
// → 删除后如果 entries 为空对象，也一并删除
```

> **衍生解释**： `__proto__` 、 `prototype` 、 `constructor` 是 JavaScript 的原型链属性。如果不阻止这些路径，攻击者可以通过构造恶意的配置路径（如 `__proto__.polluted` ）实施**原型链污染攻击（Prototype Pollution）**——向所有 JavaScript 对象注入恶意属性。OpenClaw 通过显式阻止这些关键字来防御此类攻击。

版本戳记

每次配置写入时，会自动添加版本戳记：

```
function stampConfigVersion(cfg: OpenClawConfig): OpenClawConfig {
  return {
    ...cfg,
    meta: {
      ...cfg.meta,
      lastTouchedVersion: VERSION,   // 当前 OpenClaw 版本
      lastTouchedAt: new Date().toISOString(),  // 写入时间
    },
  };
}
```

加载时会检查这个戳记——如果配置由更新版本的 OpenClaw 写入，会发出警告。这在降级场景中很有用：用户可能不小心用新版本修改了配置，然后切回旧版本运行，戳记警告可以帮助诊断潜在的兼容性问题。

本节小结

1. **配置文件**使用 JSON5 格式，默认位于 `~/.openclaw/openclaw.json` ，支持环境变量覆盖和多代历史路径兼容。
2. **Schema 系统**由 Zod 驱动，提供编译时类型推断和运行时数据校验，同时生成 JSON Schema 供 Web UI 使用。
3. **加载流水线**包含 12 个步骤：JSON5 解析 → $include$合并 → 环境变量注入 →{VAR} 替换 → 校验 → 默认值填充 → 路径规范化 → 运行时覆盖。
4. **安全措施**贯穿整个系统：文件权限（600/700）、原子写入、备份轮转、原型链污染防护、路径穿越检测。
5. **配置缓存**默认 200ms 窗口，避免高频读取带来的磁盘 IO 开销。